

逆引き R 関数

藤野秀則,

2025-02-03

目次

1	ライブラリの読み込み	4
2	データ読み込み	4
2.1	csv ファイルを読み込みこむ.	4
2.2	データフレームオブジェクトから欠損値を含む行を一括で削除する.	4
3	データの確認	5
3.1	オブジェクトの要約情報をを表示する.	5
3.2	オブジェクトの先頭から指定した行数分を表示する.	5
3.3	オブジェクトの要約統計量を表示する.	5
3.4	データフレームに含まれる変数名 (列名) を表示する.	5
3.5	出力	6
4	基本的な処理	6
4.1	算術	6
4.1.1	絶対値を求める.	6
4.1.2	数値を四捨五入する.	6
4.1.3	平方根を求める	6
4.1.4	合計を求める	6
4.2	データの操作	7
4.2.1	ベクトルの結合	7
4.2.2	データフレームの作成	7
4.2.3	データフレームを列方向に結合する	7
4.2.4	データフレームオブジェクトを行方向に結合する	7
4.2.5	連続した数値ベクトルを生成する	8
4.2.6	ベクトルデータから指定した要素を抽出する	8
4.2.7	データフレームオブジェクトから指定した行, 列を抽出する.	8
4.2.8	データフレームオブジェクトから指定した列を抽出する.	8
4.2.9	ベクトルに含まれる要素数を求める	9
4.2.10	データフレームの行数 (サンプル数) を求める.	9

4.2.11	データフレームの列数（変数数）を求める．	9
4.3	型の変換	9
4.3.1	オブジェクトを要因型に変換する．	9
4.3.2	オブジェクトを数値型に変換する．	9
4.3.3	オブジェクトを文字列型に変換する．	9
4.3.4	行列型オブジェクトをデータフレーム型に変換する．	10
5	抽出・加工・置換・ロング-ワイドの変換	10
5.1	データの抽出	10
5.1.1	条件に合致するデータの抽出	10
5.1.2	列の抽出	10
5.2	変数の追加	11
5.3	変数の置換	11
5.4	ワイド型データからロング型データへの変換	12
5.5	ロング型データからワイド型データへの変換	12
6	データの記述統計量を得る	12
6.1	1 変数の単純な記述統計量を得る	12
6.1.1	数値ベクトルの平均値を求める．	12
6.1.2	数値ベクトルの標準偏差を求める．	13
6.1.3	数値ベクトルの分散を求める．	13
6.1.4	数値ベクトルのの最大値を求める．	13
6.1.5	数値ベクトルのの最小値を求める．	13
6.1.6	数値ベクトルのの中央値を求める．	13
6.1.7	数値ベクトルのの分位数を求める．	14
6.1.8	度数分布を求める．	14
6.1.9	1 変数の標準誤差を得る	14
6.1.10	1 変数の平均値の信頼区間を得る	14
6.2	グループごとの記述統計量を得る	15
6.2.1	1 つの変数に対して	15
6.2.2	複数の変数に対して	15
6.2.3	1 つの変数に対して、複数のグループ分け基準を指定して	15
6.2.4	複数の変数に対して、複数のグループ分け基準を指定して	15
6.2.5	複数の集計関数を同時に適用	16
7	比較をしたい	16
7.1	2 群で平均値を比較したい	16
7.1.1	データに対応関係がない場合	16
7.1.2	データに対応関係がある場合	17
7.2	2 群で分散を比較したい	17
7.3	2 群で比率を比較したい	17

7.4	2つの要因の独立性を検定したい	17
7.4.1	カイ二乗検定	18
7.4.2	Fisher の正確確率検定	18
7.5	3群以上での比較をしたい	18
8	相関を調べたい	19
8.1	相関係数を求めたい	19
8.2	相関係数の検定をしたい	19
8.3	相関の差の検定をしたい	19
8.3.1	独立サンプルにおける相関の差の検定	19
8.3.2	同一サンプルにおける1つの変数Jが共通している相関の差の検定	19
8.3.3	同一サンプルにおける互いに異なる変数間の相関の差の検定	20
9	連続変数間での予測式を作りたい・因果関係を調べたい (回帰分析)	21
9.1	単純な重回帰分析	21
9.2	標準化重回帰分析	21
9.3	交互作用項を含む重回帰分析	22
9.4	一部の変数をカテゴリカル変数として扱いたい	22
9.5	ステップワイズ法による変数選択	22
10	離散変数と連続変数の間の因果関係を調べたい (分散分析)	23
10.1	1 要因分散分析	23
10.1.1	対応なしデータ/被験者間計画	23
10.1.2	対応ありデータ/被験者内計画	23
10.1.2.1	その1	23
10.1.2.2	その2	24
10.2	2 要因分散分析	24
10.2.1	対応なしデータ/被験者間計画	24
10.2.1.1	その1	24
10.2.1.2	その2	25
10.2.2	対応ありデータ/被験者内計画	25
10.2.2.1	その1	25
10.2.2.2	その2	26
10.2.3	一方の変数だけ対応ありデータ/混合計画	26
10.2.3.1	その1	26
10.2.3.2	その2	27
11	変数を集約したい	27
11.1	主成分分析	27
11.1.1	主成分負荷量を得たい	27
11.1.2	主成分得点を得たい	27
11.2	因子分析	28

11.2.1	因子数を模索したい	28
11.2.1.1	スクリープロットを得たい	28
11.2.1.2	vss 法を実施したい	28
11.2.2	探索的因子分析を実施したい	28
11.2.3	因子分析の結果を出力させたい	29
11.2.4	因子分析の結果を図示したい	29
11.2.5	因子得点を得たい	29
12	サンプルをグルーピングしたい	30
12.1	階層的クラスタリング法	30
12.2	k-means 法	30

1 ライブラリの読み込み

- 例文: `library(tidyverse)`
- 引数: 読み込むライブラリ名
- 返回值: 特になし
- 留意点: ライブラリを読み込む前に、予めライブラリを自身の Posit Cloud にインストールする必要がある。一度インストールしておけば、その後は読み込むだけでよい。また、一度読み込んだライブラリはその実行環境上、あるいは、その Rmarkdown ファイル上ではずっと有効になる。

2 データ読み込み

2.1 csv ファイルを読み込みこむ。

- 例文: `data <- read.csv("./myfolder/data.csv", header=TRUE, encoding="UTF-8")`
- 引数:
 - 第 1 引数: ファイルのパス名. Posit Cloud 上でフォルダを作って、フォルダの中に CSV ファイルがある場合には `"/フォルダ名/ファイル名.csv"` と指定する。
 - 第 2 引数 `header=TRUE`: ヘッダーがある場合に指定
 - 第 3 引数 `encoding="UTF-8"`: 文字コードを指定. UTF-8 で文字化けしてうまく読み込めない場合には `"SJIS"` を試す。
- 返回值: 読み込んだデータを格納したデータフレーム

2.2 データフレームオブジェクトから欠損値を含む行を一括で削除する。

- 例文: `data <- na.omit(data)`
- 引数: 欠損値を含むデータフレームオブジェクト

- 返り値: 欠損値を含まないデータフレームオブジェクト

3 データの確認

3.1 オブジェクトの要約情報を表示する．

- 例文: `summary(data)`
- 引数: 要約統計量を表示したいデータフレーム, もしくはベクトル
- 返り値: 数値の場合には, 最小値, 第 1 四分位数, 中央値, 平均値, 第 3 四分位数, 最大値が表示される. 要因型の場合には, 水準名と要素数が表示される. 文字列型の場合にはサンプル数と `character` 型であることをが表示される.
- 留意点: 与えるオブジェクトのクラスによって, 表示される情報が異なっている. 上記はあくまでデータフレームやベクトルを与えた場合の出力.

3.2 オブジェクトの先頭から指定した行数分を表示する．

- 例文: `head(data, 5)`
- 引数:
 - 第 1 引数: 表示したいデータフレーム, もしくはベクトル
 - 第 2 引数: 表示したい行数. 省略した場合には 6 行分が表示される.
- 返り値: 指定した行数分のデータフレームオブジェクト.

3.3 オブジェクトの要約統計量を表示する．

- 例文: `describe(data)`
- 必要ライブラリ: `psych`
- `summary()` と異なり, 平均値, 標準偏差などが表示される.
- 引数: 要約統計量を表示したいデータフレーム, もしくはベクトル

3.4 データフレームに含まれる変数名 (列名) を表示する．

- 例文: `colnames(data)`
- 引数: 列名を表示したいデータフレーム—返り値: データフレームに含まれる変数名のベクトル

3.5 出力

- 例文: `print(data)`
- 引数: 出力したいオブジェクト
- 留意点: 単純な数値や、ベクトル、データフレームなどは、`print` を付けなくても、オブジェクト名をそのまま実行するだけで出力されるが、オブジェクトによっては `print()` で出力する場合と、オブジェクト名をそのまま実行した場合とで結果が異なる場合がある。

4 基本的な処理

4.1 算術

4.1.1 絶対値を求める。

- 例文: `abs(data)`
- 引数: 絶対値を求めたい数値を含むオブジェクト—返り値: 絶対値

4.1.2 数値を四捨五入する。

- `round()`
- オブジェクトに含まれる小数を四捨五入をする
- 例文: `round(data, 2)`
- 引数:
 - 第1引数: 数値を含むオブジェクト
 - 第2引数: 有効にする小数点以下の桁数. この桁数までが表記される
- 返り値: 四捨五入された数値

4.1.3 平方根を求める

- 例文: `sqrt(data)`
- 引数: 平方根を求めたい数値を含むオブジェクト
- 返り値: 平方根

4.1.4 合計を求める

- `sum()`
- ベクトルオブジェクトに含まれる数値の合計値を求める。

- 例文: `sum(data)`
- 引数: 合計値を求めたい数値ベクトルのオブジェクト
- 返回值: 合計値

4.2 データの操作

4.2.1 ベクトルの結合

- 例文: `c(data1, data2)`
- 引数:
 - 第 1 引数: 結合したいベクトル
 - 第 2 引数: 結合したいベクトル
 - 以下同様に結したいベクトルを指定する.
- 返回值: 結合されたベクトル.

4.2.2 データフレームの作成

- データフレームオブジェクトの作成
- 例文: `data.frame(d1=data1, d2=data2, d3=data3)`
- 引数:
 - 第 1 引数: データフレームの 1 列目にふくめるベクトルデータ. 変数名 = ベクトルデータとするとデータフレーム上での変数名を設定できる. 変数名の設定を省略した場合には, ベクトル名が変数名となる.
 - 第 2 引数: データフレームの 2 列目にふくめるベクトルデータ.
 - 第 3 引数: データフレームの 3 列目にふくめるベクトルデータ.
 - 以下同様にデータフレームの列数分だけベクトルデータを指定する.
- 返回值: データフレームオブジェクト
- 留意点: データフレームオブジェクトを作成する際には, 各列のベクトルデータの長さが一致している必要がある.

4.2.3 データフレームを列方向に結合する

- 例文: `cbind(data1, data2)`
- 引数: 結合したいデータフレームオブジェクトー返回值: 結合されたデータフレームオブジェクト
- 留意点: 結合するデータフレームオブジェクトの行数は一致している必要がある.

4.2.4 データフレームオブジェクトを行方向に結合する

- 例文: `rbind(data1, data2)`

- 引数：結合したいデータフレームオブジェクト
- 返り値：結合されたデータフレームオブジェクト
- 留意点：結合するデータフレームオブジェクトの列数は一致している必要がある。

4.2.5 連続した数値ベクトルを生成する

- 例文: `1:10`
- 引数：連続した数値の最初の値と最後の値を指定する。
- 返り値：指定した範囲の連続した数値ベクトル

4.2.6 ベクトルデータから指定した要素を抽出する

- 例文: `data[3]`, `data[1:10]`, `data[c(1,3,5)]`, `data[-1]`, `data[c(-1,-4)]` — 引数：抽出したい要素のインデックス番号。複数のものを取り出したい場合には `c(1,3,5:10)` のようにベクトルで指定。
- 返り値：抽出した要素だけのベクトルデータ。
- 留意点：負の値を指定するとその列が除外されたものが返される。負の値と正の値を混在させることはできない。

4.2.7 データフレームオブジェクトから指定した行、列を抽出する。

- 例文: `data[3,5]`, `data[c(1,3,5:10), c(1,3,5:10)]`, `\ data[c(1,3,5:10), ], data[, c(1,3,5:10)]`, `data[c(1,3,5:10)]`
- 引数：
 - 第1引数：抽出したい行のインデックス番号。記載方法はベクトルデータの抽出と同様。省略した場合、すべての行が抽出される
 - 第2引数：抽出したい列のインデックス番号。記載方法はベクトルデータの抽出と同様。省略した場合、すべての列が抽出される。また、`c("変数名1", "変数名2", "変数名3")` のように各列の変数名を指定することもできる。
- 返り値：抽出した行と列のデータフレームオブジェクト。— 留意点：`data_res<-data[c(1,3,5:10)]` という形で列だけを指定することもできる。この場合、すべての行が抽出される。

4.2.8 データフレームオブジェクトから指定した列を抽出する。

- 例文: `data$変数名`
- 引数：抽出したい列の変数名。
- 返り値：抽出した列のベクトルデータ。

4.2.9 ベクトルに含まれる要素数を求める

- 例文: `length(data)`
- 引数: 要素数を求めたいベクトルデータ
- 返回值: ベクトルに含まれる要素数

4.2.10 データフレームの行数（サンプル数）を求める．

- 例文: `nrow(data)`
- 引数: 行数を求めたいデータフレームオブジェクト
- 返回值: データフレームオブジェクトの行数（サンプル数）

4.2.11 データフレームの列数（変数数）を求める．

- 例文: `ncol(data)`
- 引数: 列数を求めたいデータフレームオブジェクト
- 返回值: データフレームオブジェクトの列数（変数数）

4.3 型の変換

4.3.1 オブジェクトを要因型に変換する．

- 例文: `as.factor(data$var1)`
- 引数: 要因型に変換したいオブジェクト
- 返回值: 要因型に変換されたオブジェクト

4.3.2 オブジェクトを数値型に変換する．

- 例文: `as.numeric(data$var1)`
- 引数: 数値型に変換したいオブジェクト
- 返回值: 数値型に変換されたオブジェクト

4.3.3 オブジェクトを文字列型に変換する．

- 例文: `as.character(data$var1)`
- 引数: 文字列型に変換したいオブジェクト
- 返回值: 文字列型に変換されたオブジェクト

4.3.4 行列型オブジェクトをデータフレーム型に変換する。

- 例文: `as.data.frame(data)`
- 引数: データフレーム型に変換したい行列型オブジェクト。
- 戻り値: データフレーム型に変換されたオブジェクト。

5 抽出・加工・置換・ロング-ワイドの変換

以下では、いずれも実行に当たっては当たっては、`library(tidyverse)` を読み込んでおくことを前提とする。

5.1 データの抽出

5.1.1 条件に合致するデータの抽出

- 例文: `filter(data, var1>10)`
- 引数:
 - 第1引数: 抽出したいデータフレームオブジェクト
 - 第2引数: 抽出条件を指定する。条件指定の主な書き方は以下の通り。複数の条件を指定する場合、`&` を使って条件をつなげると**すべての条件を満たすデータ**が、`|` を使って条件をつなげると**いずれかの条件を満たすデータ**が抽出される。
 - * `var1>10`: 指定した値より大きいデータを抽出
 - * `var1>=10`: 指定した値以上のデータを抽出
 - * `var1<10`: 指定した値より小さいデータを抽出
 - * `var1<=10`: 指定した値以下のデータを抽出
 - * `var1==10`: 指定した値と等しいデータを抽出。文字列型や要因型でも利用可能。
 - * `var1!=10`: 指定した値と異なるデータを抽出。文字列型や要因型でも利用可能。
 - * `var1 %in% c(1,2,3)`: 指定した値のいずれかに合致するデータを抽出。要因型でも利用可能。
- 戻り値: 指定した条件で抽出されたデータフレームオブジェクト

5.1.2 列の抽出

- 例文: `select(data, var1, var2)`
- 引数:
 - 第1引数: 抽出したいデータフレームオブジェクト
 - 第2引数: 抽出したい列を指定する。`-` を使って列を指定すると、その列を除外することができる。
- 戻り値: 指定した列で抽出されたデータフレームオブジェクト
- 留意点: `[]` を使って列を抽出するものと同じ効果を持つ。パイプライン処理をするときには `select()` を使う。

5.2 変数の追加

- 例文: `mutate(data, var3=var1+var2)`
- 引数：
 - 第 1 引数：追加したいデータフレームオブジェクト
 - 第 2 引数：追加したい列名とその計算式を指定する.
 - 第 3 引数以降：他の変数を追加したい場合には，第 2 引数に倣って順次数式を指定する.
- 返り値：新たな列が追加されたデータフレームオブジェクト
- 留意点：データフレームに含まれる変数に何らかの演算処理を行って新たなデータを作り，元のデータオブジェクトに格納する．演算処理の際に，データフレームに含まれる変数は変数名を指定するだけでよい．データフレーム外の変数を用いることもできる．

5.3 変数の置換

- 例文:

```
library(tidyverse)
data$var3<-case_when(data$var1>3 ~ "A",
                     data$var1<=3 & data$var2 == "Y" ~ "B",
                     TRUE ~ "C")
```

データフレームに含まれるデータから条件に応じて値を置換した変数を作り出す．data に含まれる var1 の値が 3 より大きい場合には”A”，var1 が 3 以下かつ var2 が”Y” の場合には”B”，それ以外の場合には”C” と置換する．置換されたデータは var3 に格納される．

- 引数：
 - 第 1 引数：置換条件を指定．~ の左側に検索条件を指定し，右側に条件を満たすものの置換値（文字列でも数値でも可）を指定する．& や | を用いて検索条件を複数並べても良い
 - 第 2 引数以降：必要であれば，他の置換条件を追加したい場合には，第 2 引数に倣って順次条件と置換値を指定する．
 - 最終引数：すべての条件に合致しない場合の置換値を指定する場合，~ の左側に TRUE を指定し，右側に置換値を指定する．もしこの指定がなかった場合，すべての条件に合致しないものは NA に置換される．
- 返り値：置換後の変数
- 留意点：case_when() は以下のように mutate() と組み合わせて使うことが多い．mutate() の中で用いる場合には，data\$ をつける必要はない．

5.4 ワイド型データからロング型データへの変換

- 例文: `pivot_longer(data, cols=c(var1, var2), names_to="変数名", values_to="値")`
- 引数:
 - 第1引数: 変換したいデータフレームオブジェクト
 - 第2引数: 1変数にまとめた複数の変数を `c()` を用いて指定する. `c(-var1, -var2)` というように-を用いて指定した場合, 指定した変数以外の変数がすべて使われたロング型データが作成される.
 - 第3引数 `names_to="変数名"`: ロング型に変換した際の変数名を指定する.
 - 第4引数 `values_to="値"`: ロング型に変換した際の値が格納される変数名を指定する.
- 返回值: ロング型に変換されたデータフレームオブジェクト.
- 留意点: 指定していない変数は同じ値が繰り返される. 例えば, 以下の例では, `id` と性別は `変数 = 変数 1` と `変数 = 変数 2` とで同じ値が繰り返される.

5.5 ロング型データからワイド型データへの変換

- 例文: `pivot_wider(data, names_from="変数名", values_from="値")`
- 引数:
 - 第1引数: 変換したいデータフレームオブジェクト
 - 第2引数 `names_from="変数名"`: ワイド型に変換した際の変数名を指定する.
 - 第3引数 `values_from="値"`: ワイド型に変換した際の値を指定する.
- 返回值: ワイド型に変換されたデータフレームオブジェクト

6 データの記述統計量を得る

6.1 1変数の単純な記述統計量を得る

6.1.1 数値ベクトルの平均値を求める.

- 例文: `mean(data, na.rm=TRUE)`
- 引数:
 - 第1引数: 数値ベクトル
 - 第2引数 `na.rm=TRUE`: 欠損値を除外する. 省略した場合 `FALSE` となり, 欠損値が含まれる場合には `NA` が返される.

6.1.2 数値ベクトルの標準偏差を求める．

- `sd()`
- 例文: `sd(data, na.rm=TRUE)`
- 引数:
 - 第 1 引数：数値を含むオブジェクト
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する．省略した場合 `FALSE` となり，欠損値が含まれる場合には `NA` が返される．

6.1.3 数値ベクトルの分散を求める．

- 例文: `var(data, na.rm=TRUE)`
- 引数:
 - 第 1 引数：数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する．省略した場合 `FALSE` となり，欠損値が含まれる場合には `NA` が返される．

6.1.4 数値ベクトルのの最大値を求める．

- 例文: `max(data)`
- 引数:
 - 第 1 引数：数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する．省略した場合 `FALSE` となり，欠損値が含まれる場合には `NA` が返される．

6.1.5 数値ベクトルのの最小値を求める．

- 例文: `min(data)`
- 引数:
 - 第 1 引数：数値ベクトル
 - 第 2 引数 `na.rm=TRUE`: 欠損値を除外する．省略した場合 `FALSE` となり，欠損値が含まれる場合には `NA` が返される．

6.1.6 数値ベクトルのの中央値を求める．

- 例文: `median(data)`
- 引数:
 - 第 1 引数：数値ベクトル

- 第 2 引数 `na.rm=TRUE`: 欠損値を除外する。省略した場合 `FALSE` となり、欠損値が含まれる場合には `NA` が返される。

6.1.7 数値ベクトルの分位数を求める。

- 例文: `quantile(data, c(0.25, 0.75))`
- 引数:
 - 第 1 引数: 分位数を求めたい数値を含むオブジェクト
 - 第 2 引数: 分位数の位置を 0 から 1 で指定する。複数のものを同時に得たい場合には `c(0.25, 0.75)` のように指定する。
- 返回值: 指定した分位数の値を含むベクトル (名前付き)。

6.1.8 度数分布を求める。

- 例文: `table(data), table(data$factor1, data$factor2)`
- 引数: 含まれている値の頻度を求めたいオブジェクト
- 返回值: 水準ごとの頻度を含むテーブルオブジェクト。
- 留意点: 引数に複数個の変数を指定した場合、それらのクロス集計表が作成される。

6.1.9 1 変数の標準誤差を得る

`sd()` を用いて標準偏差を求め、その値を**標本数の平方根**で割ることで求める。

```
num <- length(data)
data_res <- sd(data) / sqrt(num)
```

6.1.10 1 変数の平均値の信頼区間を得る

- 例文: `t.test(data, conf.level = 0.95)$conf.int`
- 引数:
 - 第 1 引数: 数値を含むオブジェクト
 - 第 2 引数 `conf.level=0.95` (省略可): 信頼区間の信頼係数を指定する。省略した場合は 0.95。
- 返回值: 信頼区間の値
- 留意点: `t.test()` を用いて **1 変数の t 検定** (母平均が指定した値 (デフォルトでは 0) でないかの検定) を行った後、出力されるオブジェクトに含まれる信頼区間 `conf.int` の値を `$` で得る。

6.2 グループごとの記述統計量を得る

`aggregate()` を用いる.

6.2.1 1 つの変数に対して

- 例文: `aggregate(var1 ~ group, data, mean)`
- 引数:
 - 第 1 引数: ~ の左側には集計したい変数 (`var1`) を, 右側にはグループ分けのための要因型変数 (`group`) を指定する.
 - 第 2 引数: データフレームオブジェクト.
 - 第 3 引数: 集計関数を指定する. `mean` や `sd` など.

6.2.2 複数の変数に対して

- 例文: `aggregate(cbind(var1, var2) ~ group, data, mean)`
- 引数:
 - 第 1 引数: ~ の左側に集計したい複数の変数を `cbind()` を用いて指定する. 右側にはグループ分けのための要因型を指定する.
 - 第 2 引数: データフレームオブジェクト.
 - 第 3 引数: 集計関数を指定する. `mean` や `sd` など.
- 返回值: グループごとに集計された各変数の結果がデータフレームとして返される.

6.2.3 1 つの変数に対して, 複数のグループ分け基準を指定して

- 例文: `aggregate(var1 ~ group1 + group2, data, mean)`
- 引数:
 - 第 1 引数: ~ の左側に集計したい変数を指定する. 右側にはグループ分けのため複数の要因型変数を + を用いて並べる.
 - 第 2 引数: データフレームオブジェクト.
 - 第 3 引数: 集計関数を指定する. `mean` や `sd` など.
- 返回值: 複数のグループでクロス集計された結果がロング型のデータフレームとして返される.

6.2.4 複数の変数に対して, 複数のグループ分け基準を指定して

- 例文: `aggregate(cbind(var1, var2) ~ group1 + group2, data, mean)`
- 引数:

- 第 1 引数: ~ の左側に集計したい複数の変数を `cbind()` を用いて指定する。右側にはグループ分けのため複数の要因型変数を + を用いて並べる。
- 第 2 引数: データフレームオブジェクト。
- 第 3 引数: 集計関数を指定する。 `mean` や `sd` など。
- 返り値: 複数のグループでクロス集計された結果がロング型のデータフレームとして返される。

6.2.5 複数の集計関数を同時に適用

- 例文: `aggregate(var1 ~ group, data, function(x) c(mean=mean(x), sd=sd(x)))`
- 引数:
 - 第 1 引数: ~ の左側には集計したい変数名を、右側にはグループ分けのための要因型を指定する。`cbind()` を使って複数の変数を指定することも可能。
 - 第 2 引数: データフレームオブジェクトを指定する。+ を使って複数の変数を指定することも可能。
 - 第 3 引数: `function(x)` と置いたうえで半角スペースを置き、`c()` で集計関数を与える。この際、名前付きベクトルで与えると、出力結果にその名称が反映される。

7 比較をしたい

7.1 2 群で平均値を比較したい

`t.test()` を用いる。データの対応関係の有無で使い方が異なる。

7.1.1 データに対応関係がない場合

- 例文: `t.test(x=data1, y=data2)`
- 引数:
 - 第 1 引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第 2 引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
 - 第 3 引数 `alternative`: 省略可。省略した場合、両側検定を行う。片側検定を行いたい場合には `"greater"` または `"less"` を指定する。
- 返り値: 検定結果を含むリストオブジェクト。 `print()` で出力させることができる。

ロング型データの場合、次のように記述することもできる。

- 例文: `t.test(var~group, data)`
- 引数:
 - 第 1 引数: 数値変数とグループ変数 ~ で結んで指定する。グループ変数は要因型で水準は 2 つだけであることが前提。
 - 第 2 引数: データフレームオブジェクト

7.1.2 データに対応関係がある場合

- 例文: `t.test(x=data1, y=data2, paired=TRUE)`
- 引数:
 - 第 1 引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第 2 引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
 - 第 3 引数 `alternative`: 省略可. 省略した場合, 両側検定を行う. 片側検定を行いたい場合には `"greater"` または `"less"` を指定する.
 - 第 4 引数 `paired=TRUE`: 対応関係がある場合には `TRUE` を指定する.
- 留意点: 対応関係がある場合, データの数は等しくなければならない. また, 対応関係のあるデータの場合, ロング型データを用いることはできない. あくまで `filter()` などを用いて, それぞれのデータを別個のベクトルとして抽出してから `t.test()` を適用する.

7.2 2 群で分散を比較したい

- 例文: `var.test(x=data1, y=data2)`
- 引数:
 - 第 1 引数 `x`: 比較したい数値ベクトル 1. `x=` は省略しても良い
 - 第 2 引数 `y`: 比較したい数値ベクトル 2. `y=` は省略しても良い
- 返回值: 検定結果を含むリストオブジェクト. `print()` で出力させることができる.

7.3 2 群で比率を比較したい

- 例文: `prop.test(c(10, 20), c(100, 200))`
- 引数:
 - 第 1 引数: それぞれの群での成功数 (分子に来る数) をベクトルで指定する.
 - 第 2 引数: それぞれの群での総数 (分母に来る数) をベクトルで指定する.

予め `table()` を用いてクロス集計表を作成しておくと, それを引数として `prop.test()` を適用することもできる.

- 例文: `prop.test(data_table)`
- 引数: `table()` で作成したクロス集計表オブジェクト. 関数の中で `table()` を使ってもよい.

7.4 2 つの要因の独立性を検定したい

クロス集計表を作成したときに, いずれかのセルが 5 以下になっている場合には, Fisher の正確確率検定を行う. それ以外の場合には, カイ二乗検定を行う (Fisher の正確確率検定を使っても良いが, 計算時間が長くな

ってしまったり、処理がタイムアウトしてしまう可能性がある)。

7.4.1 カイ二乗検定

- 例文: `chisq.test(data_table, correct=FALSE)`
- 引数:
 - 第 1 引数: `table()` を用いて作成したクロス集計表オブジェクト。 `chisq.test()` の中で `table()` を使ってもよい。
 - 第 2 引数 `correct=FALSE`: Yates の補正を行うかどうか。いずれかのセルが 5 以下の場合には TRUE にするが、最近ではその場合は Fisher の正確確率検定を用いるのが一般的。逆に 5 以上の場合には必ず FALSE にする。

`table()` を介さず、それぞれの要因のベクトルデータを直接 `chisq.test()` に与えても良い。

- 例文: `chisq.test(data$factor1, data$factor2, correct=FALSE)`
- 引数:
 - 第 1 引数: 要因 1 のベクトルデータ
 - 第 2 引数: 要因 2 のベクトルデータ
 - 第 3 引数 `correct=FALSE`: Yates の補正を行うかどうか。上記参照。

7.4.2 Fisher の正確確率検定

f- 例文: `fisher.test(data_table)` - 引数: - 第 1 引数: `table()` を用いて作成したクロス集計表オブジェクト。 `fisher.test()` の中で `table()` を使ってもよい。

7.5 3 群以上での比較をしたい

平均の比較にせよ、分散の比較にせよ、比率の比較にせよ、3 群以上の間で比較をしたい場合には、予めすべての組み合わせでの一対比較を行ない、p 値を取得したうえで、`p.adjust()` を用いて多重比較の補正を行う。

- 例文: `p.adjust(c(p1, p2, p3), method="holm")`
- 引数:
 - 第 1 引数: 比較したい p 値をベクトルで指定する。
 - 第 2 引数 `method`: 多重比較の補正方法を指定する。 "holm" や "bonferroni" など。
- 留意点: p 値はいずれの検定の場合も、出力されるオブジェクトに `$p.value` でアクセスすることで取得できる。

8 相関を調べたい

8.1 相関係数を求めたい

- 例文: `cor(data1, data2)`, `cor(data)`
- 引数:
 - 第 1 引数: 数値ベクトル 1, もしくは数値のデータフレーム
 - 第 2 引数: 数値ベクトル 2
- 返回值: 相関係数を返す.

8.2 相関係数の検定をしたい

- 例文: `cor.test(data1, data2)`
- 引数:
 - 第 1 引数: 数値ベクトル 1
 - 第 2 引数: 数値ベクトル 2
- 返回值: 相関係数の検定結果を含むリストオブジェクト.

8.3 相関の差の検定をしたい

8.3.1 独立サンプルにおける相関の差の検定

- 例文: `cocor.indep.groups(r1, r2, n1, n2)`
- `cocor` パッケージを用いる.
- 引数:
 - 第 1 引数: 相関係数 1
 - 第 2 引数: 相関係数 2
 - 第 3 引数: サンプルサイズ 1
 - 第 4 引数: サンプルサイズ 2
- 返回值: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は中段の `fisher1925` の部分にあり, この中の `p-value` が有意確率である.
- Fisher の Z 検定

8.3.2 同一サンプルにおける 1 つの変数 J が共通している相関の差の検定

- 例文: `cocor.dep.groups.overlap(r.jk, r.jh, r.hk, n)`
- `cocor` パッケージを用いる.

- 引数：
 - 第 1 引数 r_{jk} : 相関係数比較元
 - 第 2 引数 r_{jh} : 相関係数比較先
 - 第 3 引数 r_{hk} : 共通していない変数同士の相関係数
 - 第 4 引数: サンプルサイズ
- 返り値: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は `steiger1980` の結果を見るのが一般的である (この検定のことを Stieger の Z 検定と呼ぶ)

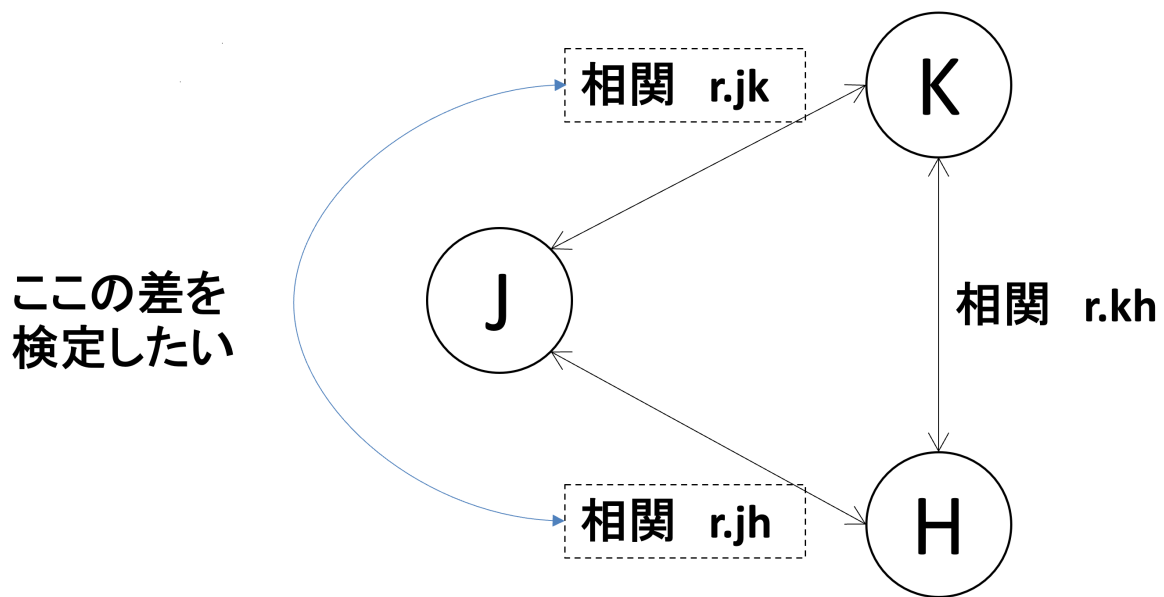


図1 1つの変数が共通している相関の差の検定

8.3.3 同一サンプルにおける互いに異なる変数間の相関の差の検定

- 例文: `cocor.dep.groups.nonoverlap(r.jk, r.hm, r.jh, r.jm, r.kh, r.km n)`
- `cocor` パッケージを用いる.
- 引数：
 - 第 1 引数 r_{jk} : 1 つ目 (J-K) の相関係数
 - 第 2 引数 r_{hm} : 2 つ目 (H-M) の相関係数
 - 第 3 引数 r_{jh} : J-H 間の相関係数
 - 第 4 引数 r_{jm} : J-M 間の相関係数
 - 第 5 引数 r_{kh} : K-H 間の相関係数
 - 第 6 引数 r_{km} : K-M 間の相関係数
 - 第 7 引数 n : サンプルサイズ
- 返り値: 相関係数の差の検定結果を含むリストオブジェクト. 検定結果は `steiger1980` の結果を見るのが一般的である (この検定のことを Stieger の Z 検定と呼ぶ)

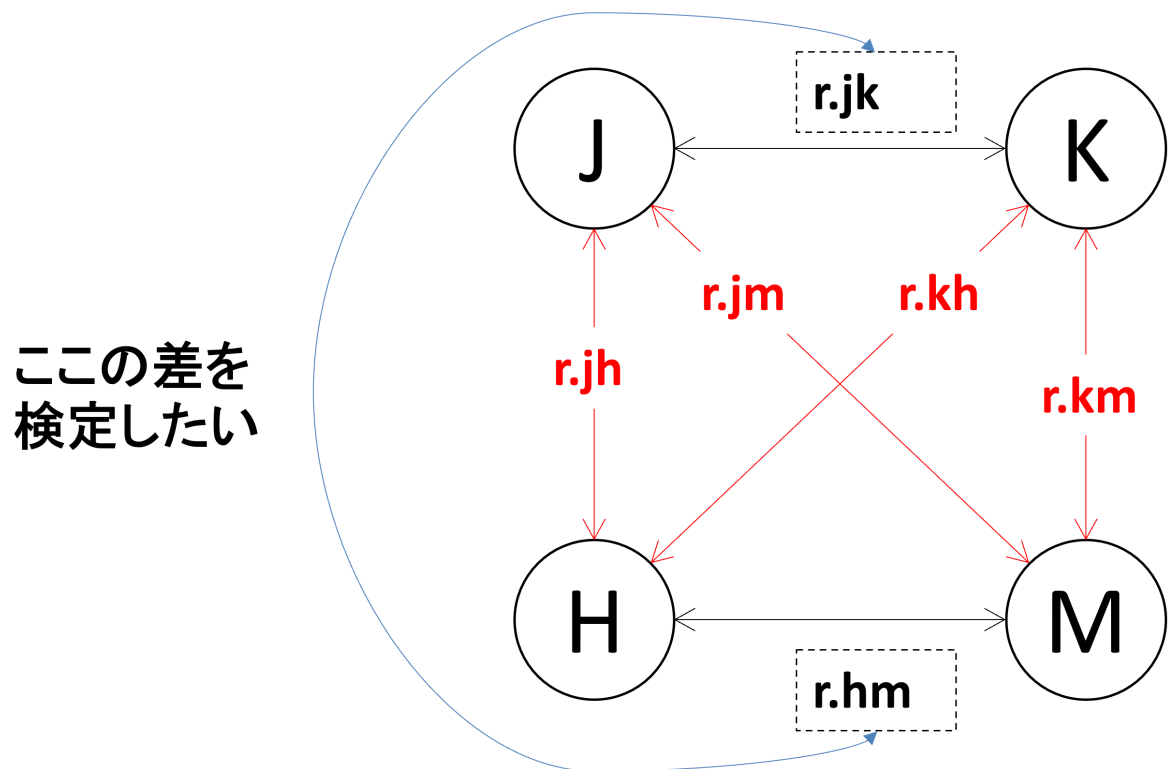


図2 互いに異なる変数間の相関の差の検定

9 連続変数間での予測式を作りたい・因果関係を調べたい (回帰分析)

9.1 単純な重回帰分析

- 例文: `lm(y ~ x1 + x2, data)`
- 引数:
 - 第1引数: 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する.
 - 第2引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト. `summary()` で出力させることができる.

9.2 標準化重回帰分析

- 例文: `lm(scale(y) ~ scale(x1) + scale(x2), data)`
- 引数:
 - 第1引数: 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する. このとき, 標準化したい変数には `scale()` を適用すると, その変数が標準化された上で回帰分析が掛けられる.
 - 第2引数: データフレームオブジェクト

- 返回值: 回帰分析の結果を含むオブジェクト. `summary()` で出力させることができる.
- 留意点: 全変数を標準化する場合には, データそのものに `scale()` を適用することもできる. ただし, その場合には `scale()` の返回值が行列型になるため, データフレームオブジェクトとして扱うためには `as.data.frame()` を適用する必要がある.

9.3 交互作用項を含む重回帰分析

- 例文: `lm(y ~ x1 * x2, data)`
- 引数:
 - 第 1 引数: 目的変数 (従属変数) と説明変数 (独立変数) を ~ で結んで指定する. 交互作用項を含む場合には * を用いて指定する. なお, 交互作用と主効果を明確に分けて表現したい場合には, `x1+x2+x1:x2` と記述すれば良い.
 - 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト. `summary()` で出力させることができる.

9.4 一部の 변수をカテゴリカル変数として扱いたい

- 例文: `lm(y ~ factor(x1) + x2, data)`
- 引数:
 - 第 1 引数: 目的変数 (従属変数) と説明変数 (独立変数) を ~ で結んで指定する. カテゴリカル変数には `factor()` を適用する.
 - 第 2 引数: データフレームオブジェクト
- 返回值: 回帰分析の結果を含むオブジェクト. `summary()` で出力させることができる.

9.5 ステップワイズ法による変数選択

- 例文: `step(res.lm, trace=FALSE)`
- 引数:
 - 第 1 引数: 回帰分析の結果を含むオブジェクト.
 - 第 2 引数 `trace` (省略化): 途中経過を表示するかどうか. `FALSE` で非表示となる. 省略した場合は `TRUE` となり, 変数選択の過程がすべて表示される.
 - 第 3 引数 `scope=list(lower=)` (省略可): 変数選択の範囲を指定する. `lower` には最小モデル (削除してほしくない変数のみで実施された `lm()` の結果) を指定する.
 - 第 4 引数 `direction` (省略化): 変数選択の方向を指定する. "both" で前進選択と後退選択を同時に行う. 他には "forward" (前進のみ) や "backward" (後退のみ) がある. 省略すると "both" となる.
- 返回值: ステップワイズ法による変数選択の結果を含む `lm` オブジェクト. `summary()` で出力させるこ

とができる。

- 留意点：変数を入れたり外したりしながら `lm()` を繰り返し実施し、AIC が最も低くなるようなモデルを見つけ出す。あくまで理論は一切考慮せずに結果だけを見て変数選択するため、その結果が理論的に良いモデルであるとは限らない。

10 離散変数と連続変数の間の因果関係を調べたい（分散分析）

10.1 1 要因分散分析

10.1.1 対応なしデータ/被験者間計画

- 例文: `aov(y ~ x, data)`
- 引数：
 - 第 1 引数: モデル式。目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。説明変数は要因型である必要がある。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 分散分析の結果を含むオブジェクト。 `summary()` で出力させることができる。

10.1.2 対応ありデータ/被験者内計画

2つの書き方がある。

■10.1.2.1 その1

- 例文: `aov(y ~ x + Error(subject/x), data)`
- 引数：
 - 第 1 引数: モデル式。目的変数（従属変数）と説明変数（独立変数）を `~` で結んで指定する。各項の説明は以下の通り。
 - * `y`: 目的変数（連続変数）
 - * `x`: 説明変数（要因型変数）
 - * `Error(subject/x)`: `subject`（被験者 ID もしくはサンプル ID.）の違いによる影響（`Subject` の主効果, `Sucject` と `x` の交互作用）を誤差として扱うことを指示する項。
 - ・ もし `x` の各水準で各 `Sucject` の測定が 1 回しかない場合には、`Sucject` と `x` の交互作用は検討できないので `Error(subject)` と記述しても同じ結果が得られる。
 - 第 2 引数: データフレームオブジェクト
- 返回值: 分散分析の結果を含むオブジェクト。 `summary()` で出力させることができる。

■10.1.2.2 その2

- 例文: `aov_ez(id=subject, dv=y, within=x, data=data)`
- `afex` パッケージを用いる.
- 引数:
 - 第1引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第2引数 `dv`: 目的変数 (従属変数)
 - 第3引数 `within`: 説明変数 (要因型変数)
 - 第4引数 `data`: データフレームオブジェクト
 - 第5引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

10.2 2 要因分散分析

10.2.1 対応なしデータ/被験者間計画

2つの要因が共に被験者間計画になっている場合である. 2つの方法がある..

■10.2.1.1 その1 `aov()` と `Anova()` を組み合わせて行う.

- 例文:

```
library(car)
res.aov <- aov(y ~ x1*x2, data)
Anova(res.aov, type=3)
```

- `car` パッケージを用いる (`Anova()` に対して).
- 引数:
 - `aov()` 部分について
 - * 第1引数: モデル式. 目的変数と2つの説明変数 (共に**要因型**) を ~ で結んで指定する. 交互作用項を含む場合には両説明変数 * を用いて並べるか, + で並べたあとに `x1:x2` という交互作用項を置く.
 - ・ `y`: 目的変数 (連続変数)
 - ・ `x1`: 説明変数 (要因型変数) その1
 - ・ `x2`: 説明変数 (要因型変数) その2
 - ・ `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要.

- * 第 2 引数: データフレームオブジェクト
- `Anova()` 部分について
 - * 第 1 引数: `aov()` の結果を含むオブジェクト
 - * 第 2 引数 `type=3`: 交互作用項を持つ 2 要因分散分析を行う場合には `type=3` を指定する. 交互作用をおかない場合には `type=2` を指定する.
- 戻り値: 結果を含むオブジェクト.
- 留意点: 2 要因分散分析を行う場合には, `aov()` の結果を `Anova()` に与えることで, 2 要因分散分析を行うことができる.

■10.2.1.2 その 2

- 例文: `aov_ez(id=subject, dv=y, between=c(x1, x2), data=data, type=3)`
- `afex` パッケージを用いる.
- 引数:
 - 第 1 引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第 2 引数 `dv`: 目的変数 (従属変数)
 - 第 3 引数 `between`: 2 つの説明変数 (要因型変数) を `c()` で括って指定する.
 - 第 4 引数 `data`: データフレームオブジェクト
 - 第 5 引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 戻り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.
- 留意点: `aov_ez()` を用いる場合, サンプル識別番号 `subject` (要因型) を設ける必要がある. 被験者間計画なので, サンプル識別番号はあくまで 1 つのデータと 1:1 で対応することになる.

10.2.2 対応ありデータ/被験者内計画

2 つの要因が共に被験者内計画になっている場合である. 2 つの方法がある.

■10.2.2.1 その 1

- 例文: `aov(y ~ x1*x2 + Error(subject/(x1*x2)), data)`
- 引数:
 - 第 1 引数: モデル式. 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する. 各項の説明は以下の通り.
 - * `y`: 目的変数 (連続変数)
 - * `x1`: 説明変数 (要因型変数) その 1
 - * `x2`: 説明変数 (要因型変数) その 2
 - * `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要.

- * `Error(subject/(x1*x2))`: subject (被験者 ID もしくはサンプル ID.) の違いによる影響 (Subject の主効果, Subject と x の交互作用,, Subject と y の交互作用, Subject と x と y の交互作用) を誤差として扱うことを指示する項.
- 第 2 引数: データフレームオブジェクト
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

■10.2.2.2 その 2

- 例文: `aov_ez(id=subject, dv=y, within=c(x,x2), data=data, type=3)`
- `afex` パッケージを用いる.
- 引数:
 - 第 1 引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第 2 引数 `dv`: 目的変数 (従属変数)
 - 第 3 引数 `within`: 2つの説明変数 (要因型変数) を `c()` で括って指定する.
 - 第 4 引数 `data`: データフレームオブジェクト
 - 第 5 引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

10.2.3 一方の変数だけ対応ありデータ/混合計画

2つの要因のうち1つは被験者間計画, もう一つが被験者内計画になっている場合である. 2つの方法がある.

■10.2.3.1 その 1

- 例文: `aov(y ~ x1*x2 + Error(subject/x2), data)`
- 引数:
 - 第 1 引数: モデル式. 目的変数 (従属変数) と説明変数 (独立変数) を `~` で結んで指定する. 各項の説明は以下の通り.
 - * `y`: 目的変数 (連続変数)
 - * `x1`: 被験者間計画の説明変数 (要因型変数)
 - * `x2`: 被験者内計画の説明変数 (要因型変数)
 - * `x1:x2`: 交互作用項. `x1*x2` と記述した場合には記述不要.
 - * `Error(subject/x2)`: subject (被験者 ID もしくはサンプル ID.) の違いによる影響 (Subject の主効果, Subject と x の交互作用,, Subject と y の交互作用, Subject と x と y の交互作用) を誤差として扱うことを指示する項.
 - 第 2 引数: データフレームオブジェクト
- 返り値: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

■10.2.3.2 その2

- 例文: `aov_ez(id=subject, dv=y, between=x1, within=x2, data=data, type=3)`
- `afex` パッケージを用いる.
- 引数:
 - 第1引数 `id`: 被験者 ID もしくはサンプル ID.
 - 第2引数 `dv`: 目的変数 (従属変数)
 - 第3引数 `between`: 被験者間計画の説明変数 (要因型変数)
 - 第4引数 `within`: 被験者内計画の説明変数 (要因型変数)
 - 第5引数 `data`: データフレームオブジェクト
 - 第6引数 `type=3`: 分散分析の種類. "3" で被験者内計画の分散分析を行う
- 返回值: 分散分析の結果を含むオブジェクト. `summary()` で出力させることができる.

11 変数を集約したい

11.1 主成分分析

- 例文: `prcomp(data, scale=TRUE)`
- 引数:
 - 第1引数: データフレームオブジェクト
 - 第2引数 `scale=TRUE` (省略可): データを標準化するかどうか. 標準化したくない場合には `FALSE` を指定する. 省略すると標準化が行われる.
- 返回值: 主成分分析の結果を含むオブジェクト. 各主成分の分散説明率や累積説明率は `summary()` で出力させることができる.

11.1.1 主成分負荷量を得たい

- 例文: `res.pca$rotation`
- 引数: 主成分分析の結果を含むオブジェクトに `$` をつけて `rotation` を指定する.
- 返回值: 主成分負荷量を含む行列オブジェクト.

11.1.2 主成分得点を得たい

- 例文: `res.pca$x`
- 引数: 主成分分析の結果を含むオブジェクトに `$` をつけて `x` を指定する.
- 返回值: 主成分得点を含む行列オブジェクト.

11.2 因子分析

11.2.1 因子数を模索したい

■11.2.1.1 スクリープロットを得たい

- 例文: `fa.parallel(data, fa="fa")`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数 `fa="fa"` (省略可): 出力を因子分析用にするか, 主成分分析用にするかを指定する. 因子分析用にする場合には `"fa"` を指定する. 主成分分析用にする場合には `"pc"` を指定する. 省略すると両方が表示される.
- 返り値: スクリープロットが出力されるとともに, 因子数の候補が表示される.

■11.2.1.2 vss 法を実施したい

- 例文: `vss(data)`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数 `n=8` (省略可): シミュレートするときの最大因子数を指定する. 省略すると 8 が指定される.
- 返り値: VSS の結果が表示されるとともに, VSS のグラフが出力される.

11.2.2 探索的因子分析を実施したい

- 例文: `fa(data, 2, rotate="oblimin", fm="minres")`
- `psych` パッケージを用いる.
- 引数:
 - 第 1 引数: データフレームオブジェクト
 - 第 2 引数: 因子数を指定する.
 - 第 3 引数 `rotate="oblimin"` (省略可): 因子負荷量の回転方法を指定する. 省略すると斜交回転の 1 種である `oblimin` が行われる. 他に, 直交回転系として `"varimax"`, `"quartimax"`, 斜交回転系で `"promax"` などが指定できる.

- 第4引数 `fm="minres"` (省略可): 因子分析の方法を指定する. "minres" で最小残差法が指定される. 他には"ml" (最尤法) や"pa" (主軸法) などが指定できる.
- 返り値: 因子分析の結果を含むオブジェクト. 出力は以下を参照.

11.2.3 因子分析の結果を出力させたい

- 例文: `print(res.fa, sort=TRUE, digits=2)`
- `psych` パッケージを用いる.
- 引数:
 - 第1引数: 因子分析の結果を含むオブジェクト
 - 第2引数 `sort=TRUE` (省略可): 因子負荷量を表示する際に, 各因子の因子負荷量の大きい順に表示するかどうか. `TRUE` にすると各因子ごとに負荷量の大きい順に表示される. 省略すると `FALSE` となり, 変数名の順に表示される.
 - 第3引数 `digits=2` (省略可): 因子負荷量の表示桁数を指定する. 省略すると2桁になる.
- 返り値: 因子分析の結果が表示される.

11.2.4 因子分析の結果を図示したい

- 例文: `fa.diagram(res.fa, digits=2, cut=.3, simple=TRUE)`
- `psych` パッケージを用いる.
- 引数:
 - 第1引数: 因子分析の結果を含むオブジェクト
 - 第2引数 `digits=2` (省略可): 因子負荷量の表示桁数を指定する. 省略すると2桁になる.
 - 第3引数 `cut=.3` (省略可): 因子負荷量の絶対値がこの値より小さい場合には表示しない. 省略すると0.3が指定される.
 - 第4引数 `simple=TRUE` (省略可): `TRUE` にすると各項目につき因子負荷量が最も高い因子との結びつきのみが表示される. `FALSE` にすると全ての因子との結びつきが表示する. ただしいずれも `cut` を下回るものについては表示されない.
- 返り値: 図示された因子分析の結果.

11.2.5 因子得点を得たい

- 例文: `res.fa$scores`
- 引数: 因子分析の結果を含むオブジェクトに `$` をつけて `scores` を指定する.
- 返り値: 因子得点を含む行列オブジェクト.

12 サンプルをグルーピングしたい

12.1 階層的クラスタリング法

以下の手順で実施する。

1. `scale()` を用いてデータを標準化する。
 - 引数: データフレームオブジェクト
2. `dist()` を用いてデータの距離行列を求める。
 - 第 1 引数: 標準化させたデータオブジェクト
 - 第 2 引数 `method`: 距離の計算方法を指定する。省略した場合、ユークリッド距離となる。
3. `hclust()` を用いて階層的クラスタリングを行う。
 - 第 1 引数: 距離行列オブジェクト
 - 第 2 引数 `method`: クラスタリングの方法を指定する。省略した場合には"`complete`"（最長距離法）が指定される。
4. `plot()` を用いてデンドログラムを表示させ、クラスタ数を決める。
5. `cutree()` を用いてクラスタリングの結果を取得する。
 - 第 1 引数: `hclust()` の結果を含むオブジェクト
 - 第 2 引数 `k`: クラスタ数を指定する。

例文:

```
data_std <- scale(data)
data_dist <- dist(data_std)
hc <- hclust(dist_data, method="ward.D2")
plot(hc)
res.clust<-cutree(hc, k=3)
```

- 最終的な返り値: 各サンプルに振られるクラスタ番号を収めたベクトル。これを例えば `data$cluster <- res.clust` のようにデータフレームに追加することで、クラスタリングの結果を元のデータフレームに保存することができ、クラスターごとの平均値の算出などに利用できる (参考)。

12.2 k-means 法

- 例文: `kmeans(data, centers=3, nstart=10)`
- 引数:
 - 第 1 引数: データフレームオブジェクト (標準化済みのものがよい)
 - 第 2 引数 `centers`: クラスタ数を指定する。

- 第3引数 `nstart=10`: 初期値を変えてクラスタリングを行う回数を指定する.
- 返り値: クラスタリングの結果を含むオブジェクト. `res.kmeans$cluster` で各サンプルに振られるクラスタ番号を取得できる.
- 留意点: k-means 法は初期値によってクラスタリングの結果が変わることがある. そのため, `nstart` を大きくすることで, 複数回クラスタリングを行い, 最もクラスタリングが良いものを選ぶことができる.